Bài tập Toán 03–04 — Ma trận, hệ tuyến tính và PCA

1. Đề bài tiếng Anh

1. Data Exploration and Matrix Creation: create an eight-row store matrix with Orders, Customers, Revenue, and Employees; display matrix shape, customer values, and average customers.
2. Finding Evaluation Weights Using a Linear System: solve the supplied 3 × 3 system A @ w = b for three store-evaluation weights.
3. Calculating and Ranking Store Performance: scale Orders, Customers and Revenue into comparable score units, compute S @ w, rank all eight stores and identify the best store.
4. Data Standardization and Dimensionality Reduction Using PCA: standardize the four store features, apply PCA to reduce to two components, and inspect projected shape and explained variance.

2. Đề bài tiếng Việt

1. Khám phá dữ liệu và tạo ma trận: lập bảng tám cửa hàng với Đơn hàng, Khách hàng, Doanh thu và Nhân viên; in kích thước ma trận, cột Khách hàng và số khách trung bình.
2. Tìm trọng số đánh giá bằng hệ tuyến tính: giải hệ 3 × 3 đã cho theo biểu thức A @ w = b để tính ba trọng số đánh giá.
3. Tính và xếp hạng hoạt động cửa hàng: quy đổi Đơn hàng, Khách hàng và Doanh thu thành các thang điểm tương ứng, tính S @ w, xếp tám cửa hàng và tìm cửa hàng cao điểm nhất.
4. Chuẩn hóa dữ liệu và giảm số chiều bằng PCA: chuẩn hóa bốn cột dữ liệu, đưa về hai thành phần chính, kiểm tra kích thước sau biến đổi và phần trăm phương sai được giữ lại.

3. Code Python có chú thích

```python
"""Lab Math 03-04 — store evaluation using matrices, linear systems and PCA.

Adapted from 2. Math/labs3_4.ipynb. Run:
    python SOLUTION_ANNOTATED.py

Requires numpy and scikit-learn; prints results without opening a GUI.
"""
import numpy as np  # Arrays, descriptive statistics and matrix multiplication.
from sklearn.preprocessing import StandardScaler  # Scale each feature.
from sklearn.decomposition import PCA  # Reduce correlated features to 2 axes.

# Rows A-H represent stores. Each column has a specified real-world unit:
# Orders (count), Customers (count), Revenue (currency units), Employees (count).
STORE_NAMES = np.array(list("ABCDEFGH"))
STORE_DATA = np.array([
    [300, 400, 100000, 20],
    [400, 500, 140000, 25],
    [200, 200, 70000, 15],
    [500, 600, 180000, 30],
    [350, 450, 115000, 22],
    [450, 550, 160000, 28],
    [250, 300, 85000, 18],
    [600, 700, 210000, 35],
], dtype=float)  # Shape (8,4), with numeric values compatible with NumPy.
```

```python
# A and b are supplied evaluation coefficients and observed benchmark scores.
EQUATION_MATRIX = np.array([[3, 4, 10], [4, 5, 14], [2, 2, 7]], dtype=float)
EQUATION_SCORES = np.array([135, 182, 86], dtype=float)


def task1_explore(stores=STORE_DATA):  # Tám cửa hàng, mỗi cửa hàng có bốn đặc trưng.
    """Inspect matrix dimensions, customer counts and mean customer traffic."""
    if stores.ndim != 2 or stores.shape[1] != 4:
        raise ValueError("Expect a matrix with four store features.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    customers = stores[:, 1]  # ':' means all rows; index 1 is Customers.
    mean_customers = float(np.mean(customers))  # Mean over all eight stores.
    return stores.shape, customers, mean_customers


def task2_weights(A=EQUATION_MATRIX, b=EQUATION_SCORES):  # Giải A @ w = b để tìm trọng số, không nhân từng phần tử.
    """Solve A @ w = b for weights of Orders, Customers and Revenue."""
    # np.linalg.solve avoids manually computing the matrix inverse.
    if A.shape != (3, 3) or b.shape != (3,):
        raise ValueError("Expected a 3x3 matrix and a three-element score vector.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    if np.linalg.matrix_rank(A) < 3:  # A singular matrix has no unique answer.
        raise ValueError("The benchmark system has no unique solution.")  # Báo lỗi nếu dữ liệu không đúng điều kiện.
    weights = np.linalg.solve(A, b)
    if not (np.allclose(A @ weights, b)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Reconstruct the input equations.
    return weights


def task3_rank(stores=STORE_DATA, names=STORE_NAMES, weights=None):  # Đổi thang đo rồi tính điểm và xếp hạng cửa hàng.
    """Build dimensionless components, combine weights, rank all stores."""
    if weights is None:
        weights = task2_weights()
    # Scale columns as in the original notebook before applying weights.
    components = np.column_stack((
        stores[:, 0] / 100,  # One score unit per 100 orders.
        stores[:, 1] / 100,  # One score unit per 100 customers.
        stores[:, 2] / 10000,  # One score unit per 10,000 revenue units.
    ))
    totals = components @ weights  # (8x3) @ (3,) -> eight store scores.
    ranked = sorted(zip(names.tolist(), totals.tolist()),
                    key=lambda pair: (-pair[1], pair[0]))
    winner_index = int(np.argmax(totals))  # Index of highest score.
    return components, totals, ranked, names[winner_index]


def task4_pca(stores=STORE_DATA):  # Chuẩn hóa trước khi giảm xuống hai thành phần chính.
    """Standardize four columns, then project stores onto two PCA axes."""
    # Different units (currency, people) must not bias distance by magnitude.
    scaler = StandardScaler()  # By default centers and scales each column.
    standardized = scaler.fit_transform(stores)
    # PCA finds directions containing the largest variation in scaled data.
    pca = PCA(n_components=2)  # We want a shape of (8 stores, 2 axes).
    projected = pca.fit_transform(standardized)
    retained = pca.explained_variance_ratio_  # Fraction on each retained axis.
    if not (projected.shape == (stores.shape[0], 2)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.allclose(standardized.mean(axis=0), 0, atol=1e-12)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.allclose(standardized.var(axis=0), 1, atol=1e-12)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.isfinite(projected).all()): raise ValueError("Kiểm tra dữ liệu không đạt")
    return standardized, projected, retained
```

```
def main():
    """Print intermediate results for every English task and check shapes."""
    shape, customers, mean_customers = task1_explore()
    print("Task 1 - Store matrix shape:", shape)
    print("Customers:", customers.astype(int).tolist())
    print("Mean customers:", round(mean_customers, 2))

    weights = task2_weights()
    print("Task 2 - Weights:", np.round(weights, 6).tolist())

    components, scores, ranking, best = task3_rank(weights=weights)
    print("Task 3 - Component matrix shape:", components.shape)
    print("Scores:", np.round(scores, 2).tolist())
    print("Ranking:", [(name, round(score, 2)) for name, score in ranking])
    print("Best store:", best)

    standardized, projection, ratios = task4_pca()
    print("Task 4 - Standardized shape:", standardized.shape)
    print("PCA projected shape:", projection.shape)
    print("Explained variance fractions:", np.round(ratios, 4).tolist())
    print("Total variance retained:", f"{ratios.sum() * 100:.2f}%")
    if not (len(ranking) == len(STORE_NAMES)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (projection.shape == (8, 2)): raise ValueError("Kiểm tra dữ liệu không đạt")


if __name__ == "__main__":
    main()  # Execute this demonstration only when launched as a script.
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Câu 1

`np.array` tạo ma trận có tám hàng và bốn cột nên `shape` là `(8, 4)`. `stores[:, 1]` có nghĩa là lấy **mọi hàng**, riêng cột chỉ số 1 (Customers); `np.mean` tính trung bình tám số khách.

Câu 2

`np.linalg.solve(A, b)` giải **hệ phương trình** `A @ w = b`, không phải nhân từng phần tử. Ma trận 3 × 3 chỉ có nghiệm duy nhất nếu đủ hạng; `np.linalg.matrix_rank(A)` kiểm tra điều này.

Câu 3

`np.column_stack` ghép ba cột đã chia thang 100, 100, 10000; `components @ weights` nhân ma trận 8 × 3 với vector 3 phần tử tạo tám điểm. `np.argmax` cho vị trí điểm lớn nhất; `sorted` sắp xếp cửa hàng.

Câu 4

Doanh thu tính bằng tiền lớn hơn hẳn số nhân viên, nên cần `StandardScaler()` để đưa các đặc trưng về thang đo có trung bình gần 0 và phương sai gần 1. `PCA(n_components=2)` biến dữ liệu tám hàng bốn cột thành tám hàng hai cột; `explained_variance_ratio_` cho biết tỷ lệ phương sai mỗi thành phần giữ lại. PCA không bảo đảm dự đoán hoàn hảo.

Đối chiếu nguồn và hướng dẫn chạy

Notebook PythoClass gốc có bốn tiêu đề đề bài tiếng Anh và mã minh họa. Lời giải đầy đủ bên dưới lấy từ `labs/pythoclass/LAB-MATH-03-04-matrix-pca/SOLUTION.py`, bản tái cấu trúc từ notebook, vẫn sử dụng dữ liệu 8 cửa hàng và hệ phương trình gốc.

Khác biệt trình bày: Các điều kiện `assert` có trong `SOLUTION.py` được đổi thành `if not ...: raise ValueError(...)` để người mới hiểu cách kiểm tra rõ ràng hơn; không đổi mục tiêu tính toán.

Nguồn bài tập: `PythoClass/2. Math/labs3_4.ipynb` trên repo `nguyenletrian/PythoClass`.

Thứ tự xử lý và lý do làm từng bước

1. Dữ liệu gốc là ma trận 8 × 4. Mỗi hàng mô tả một cửa hàng A–H, các cột là đơn hàng, khách hàng, doanh thu và số nhân viên. `stores[:, 1]` lấy cột khách hàng theo chỉ số 1 (chỉ số Python bắt đầu từ 0); `np.mean` lấy số khách trung bình. `.shape` giúp đối chiếu kích thước trước khi nhân ma trận hoặc PCA.

2. Tìm trọng số bằng hệ tuyến tính. `EQUATION_MATRIX` có 3 hàng và 3 cột, `EQUATION_SCORES` có 3 giá trị. `np.linalg.solve(A, b)` tìm vector `w` sao cho `A @ w = b`; toán tử `@` là tích ma trận, khác `*` là nhân từng phần tử trong NumPy. `matrix_rank(A)` kiểm tra ma trận có đủ hạng để nghiệm duy nhất tồn tại. Kiểm lại bằng `np.allclose(A @ w, b)` bởi số thực có sai số làm tròn.

3. Tính điểm cửa hàng: ba cột đơn hàng, khách hàng, doanh thu có thang rất khác nhau nên code chia lần lượt cho 100, 100 và 10.000 để tạo ma trận `components` kích thước 8 × 3. Nhân với trọng số `w` sẽ cho tám điểm số. `np.argmax` tìm vị trí cao nhất; `sorted(..., key=...)` xếp hạng từ cao xuống thấp.

4. PCA dùng bốn cột gốc. `StandardScaler().fit_transform(stores)` đưa mỗi cột về thang đo so sánh được (trung bình gần 0, phương sai gần 1). `PCA(n_components=2)` chọn hai hướng có phương sai lớn, cho ma trận kết quả 8 × 2. `explained_variance_ratio_` là tỷ lệ phương sai mà từng hướng giữ lại, và tổng của chúng cho mức thông tin biến thiên được giữ. PCA không phải thuật toán bảo đảm dự đoán đúng hay phương pháp tự chọn cửa hàng tốt nhất.

Lỗi nên tránh: lấy `S * w` thay vì `S @ w`, quên chuẩn hóa trước PCA khi đơn vị khác nhau, hoặc nhầm `explained_variance_ratio_` là xác suất dự đoán đúng.

5. Giải thích các định nghĩa

Các thuật ngữ ma trận và PCA được diễn giải bằng tình huống tám cửa hàng trước khi cần đến tên toán học.

Matrix (ma trận)

Là gì và dùng khi nào? Bảng gồm hàng và cột, để lưu nhiều đặc trưng của nhiều cửa hàng trong cùng một cấu trúc.

Ví dụ: 8 cửa hàng, 4 đặc trưng mỗi cửa hàng → ma trận 8 hàng × 4 cột.

Shape (kích thước mảng)

Là gì và dùng khi nào? Cho biết số hàng và số cột; giúp kiểm tra hai ma trận có thể nhân hay không.

Ví dụ: Ma trận trên có `shape = (8,4)`.

`@` — phép nhân ma trận

Là gì và dùng khi nào? Kết hợp từng hàng của đối tượng bên trái với các giá trị phù hợp bên phải; khác `*`, là phép nhân từng phần tử của NumPy.

Ví dụ: Ma trận 8×3 nhân vector có 3 trọng số tạo 8 điểm, mỗi điểm thuộc một cửa hàng.

Linear system (hệ phương trình tuyến tính)

Là gì và dùng khi nào? Nhiều phương trình có chung các ẩn cần tìm. `np.linalg.solve(A,b)` tìm các trọng số để A nhân trọng số bằng b.

Ví dụ: Một hệ 3 phương trình có thể xác định 3 trọng số nếu ma trận A đủ hạng.

Weight (trọng số)

Là gì và dùng khi nào? Hệ số quyết định mức đóng góp của một tiêu chí khi tính điểm tổng.

Ví dụ: Điểm = 2 × chỉ số đơn hàng + 1 × chỉ số khách hàng: trọng số lần lượt là 2 và 1.

Scaling và standardization (đổi thang và chuẩn hóa)

Là gì và dùng khi nào? Đổi thang chia số liệu theo một đơn vị quy ước. Chuẩn hóa thường lấy số đo trừ trung bình rồi chia độ lệch chuẩn, giúp các cột có thang tương đương.

Ví dụ: Doanh thu 100.000 chia 10.000 thành 10 điểm là đổi thang; `StandardScaler` dùng trung bình và độ lệch chuẩn của cả cột.

PCA (phân tích thành phần chính)

Là gì và dùng khi nào? Tìm các hướng có mức biến thiên lớn trong dữ liệu, biểu diễn thông tin bằng ít cột hơn. PCA không phải thuật toán phân loại chính xác/sai.

Ví dụ: `PCA(n_components=2)` biến bảng 8×4 thành 8×2; số cửa hàng giữ nguyên.

Explained variance ratio (tỷ lệ phương sai giữ lại)

Là gì và dùng khi nào? Cho biết mỗi hướng PCA giữ bao nhiêu phần biến thiên ban đầu. Cộng các tỷ lệ của các hướng đã chọn để được tổng phần biến thiên giữ lại.

Ví dụ: Nếu PC1 giữ 70%, PC2 giữ 20% thì cả hai giữ 90% phương sai, **không phải dự đoán chính xác 90%**.

Eigenvalue (trị riêng)

Là gì và dùng khi nào? Trong PCA, trị riêng biểu diễn độ lớn phương sai theo một hướng riêng; trị riêng cao thường ứng với hướng giữ nhiều biến thiên.

Ví dụ: Nếu các hướng có trị riêng 9 và 1, hướng 9 giải thích nhiều biến thiên hơn.